Family	Method	FID (↓)	Steps (↓)
Diffusion & Flow	DDPM (Ho et al., 2020)	3.17	1000
	DDPM++ (Song et al., 2020b)	3.16	1000
	NCSN++ (Song et al., 2020b)	2.38	1000
	DPM-Solver (Lu et al., 2022)	4.70	10
	iDDPM (Nichol & Dhariwal, 2021)	2.90	4000
	EDM (Karras et al., 2022)	2.05	35
	Flow Matching (Lipman et al., 2022)	6.35	142
	Rectified Flow (Liu et al., 2022)	2.58	127
Few-Step via Distillation	PD (Salimans & Ho, 2022)	4.51	2
	2-Rectified Flow (Salimans & Ho, 2022)	4.85	1
	DFNO (Zheng et al., 2023)	3.78	1
	KD (Luhman & Luhman, 2021)	9.36	1
	TRACT (Berthelot et al., 2023)	3.32	2
	Diff-Instruct (Luo et al., 2024a)	5.57	1
	PID (LPIPS) (Tee et al., 2024)	3.92	1
	DMD (Yin et al., 2024)	3.77	1
	CD (LPIPS) (Song et al., 2023)	2.93	2
	CTM (w/ GAN) (Kim et al., 2023)	**1.87**	2
	SiD (Zhou et al., 2024)	1.92	1
	SiM (Luo et al., 2024b)	2.06	1
	sCD (Lu & Song, 2024)	2.52	2
Few-Step from Scratch	iCT (Song & Dhariwal, 2023)	2.83	1
		2.46	2
	ECT (Geng et al., 2024)	3.60	1
		2.11	2
	sCT (Lu & Song, 2024)	2.97	1
		2.06	2
	IMM (ours)	3.20	1
		1.98	2

Table 1. CIFAR-10 results trained without label conditions.

Family	Method	FID(↓)	Steps (↓)	#Params
GAN	BigGAN (Brock, 2018)	6.95	1	112M
	GigaGAN (Kang et al., 2023)	3.45	1	569M
	StyleGAN-XL (Karras et al., 2020)	2.30	1	166M
Masked & AR	VQGAN (Esser et al., 2021)	26.52	1024	227M
	MaskGIT (Chang et al., 2022)	6.18	8	227M
	MAR (Li et al., 2024)	1.98	100	400M
	VAR-d20 (Tian et al., 2024a)	2.57	10	600M
	VAR-d30 (Tian et al., 2024a)	**1.92**	10	2B
Diffusion & Flow	ADM (Dhariwal & Nichol, 2021)	10.94	250	554M
	CDM (Ho et al., 2022b)	4.88	8100	-
	SimDiff (Hoogeboom et al., 2023)	2.77	512	2B
	LDM-4-G (Rombach et al., 2022)	3.60	250	400M
	U-DiT-L (Tian et al., 2024b)	3.37	250	916M
	U-ViT-H (Bao et al., 2023)	2.29	50	501M
	DiT-XL/2 ($w = 1.0$) (Peebles & Xie, 2023)	9.62	250	675M
	DiT-XL/2 ($w = 1.25$) (Peebles & Xie, 2023)	3.22	250	675M
	DiT-XL/2 ($w = 1.5$) (Peebles & Xie, 2023)	2.27	250	675M
	SiT-XL/2 ($w = 1.0$) (Ma et al., 2024)	9.35	250	675M
	SiT-XL/2 ($w = 1.5$) (Ma et al., 2024)	2.15	250	675M
Few-Step from Scratch	iCT (Song et al., 2023)	34.24	1	675M
		20.3	2	675M
	Shortcut (Frans et al., 2024)	10.60	1	675M
		7.80	4	675M
		3.80	128	675M
	IMM (ours) (XL/2, $w = 1.25$)	7.77	1	675M
		5.33	2	675M
		3.66	4	675M
		2.77	8	675M
	IMM (ours) (XL/2, $w = 1.5$)	8.05	1	675M
		3.99	2	675M
		2.51	4	675M
		1.99	8	675M

Table 2. Class-conditional ImageNet-256×256 results.

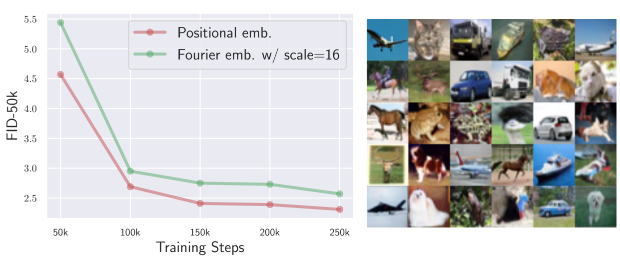

Figure 4. FID convergence for different embeddings (left). CIFAR-10 samples from Fourier embedding (scale $= 16$) (right).

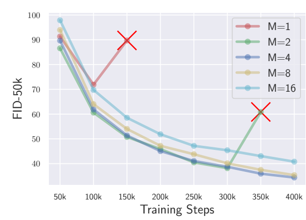

Figure 5. More particles indicate more stable training on ImageNet-256×256.

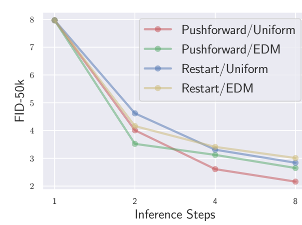

Figure 6. ImageNet-256×256 FID with different sampler types.

using pushforward sampler. For ImageNet-256×256, we use the popular DiT (Peebles & Xie, 2023) architecture because of its scalability, and compare it with GANs, masked and autoregressive models, diffusion and flow models, and few-step models trained from scratch.

We observe decreasing FID with more steps and IMM achieves 1.99 FID with 8 steps (with $w = 1.5$), surpassing DiT and SiT (Ma et al., 2024) using the same architecture except for trivially injecting time s (see Appendix I). Notably, we also achieve better 8-step FID than the 10-step VAR (Tian et al., 2024a) of comparable size. At 16 steps, IMM also achieves 1.90 FID outperforming VAR's 2B variant (see Appendix I.4). However, different from VAR, IMM grants flexibility of variable number of inference steps and the large improvement in FID from 1 to 8 steps additionally demonstrates IMM's *efficient* inferece-time scaling capability. Lastly, we similarly surpass Shortcut models' (Frans et al., 2024) best performance with only 8 steps. We defer inference details to Section 7.3 and Appendix I.2.

7.2. IMM Training is Stable

We show that IMM is stable and achieves reasonable performance across a range of parameterization choices.

Positional vs. Fourier embedding. A known issue for CMs (Song et al., 2023) is its training instability when using Fourier embedding with scale 16, which forces reliance on positional embeddings for stability. We find that IMM does not face this problem (see Figure 4). For Fourier embedding we use the standard NCSN++ (Song et al., 2020b) architecture and set embedding scale to 16; for positional embeddings, we adopt DDPM++ (Song et al., 2020b). Both embedding types converge reliably, and we include samples from the Fourier embedding model in Figure 4.

Particle number. Particle number M for estimating MMD is an important parameter for empirical success (Gretton et al., 2012; Li et al., 2015), where the estimate is more accurate with larger M. In our case, naïvely increasing M can slow down convergence because we have a fixed batch size B in which the samples are grouped into B/M groups

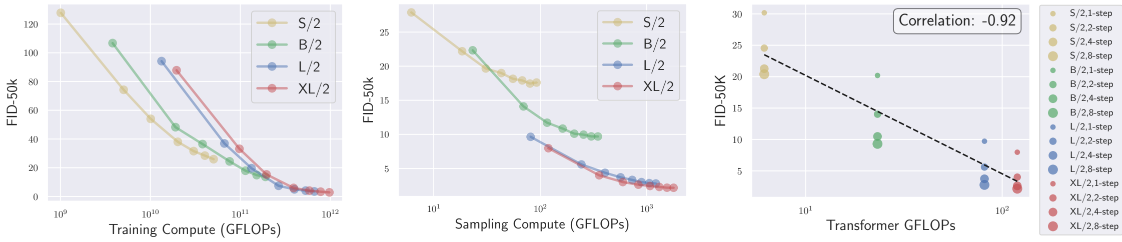

Figure 7. IMM scales with both training and inference compute, and exhibits strong correlation between model size and performance.

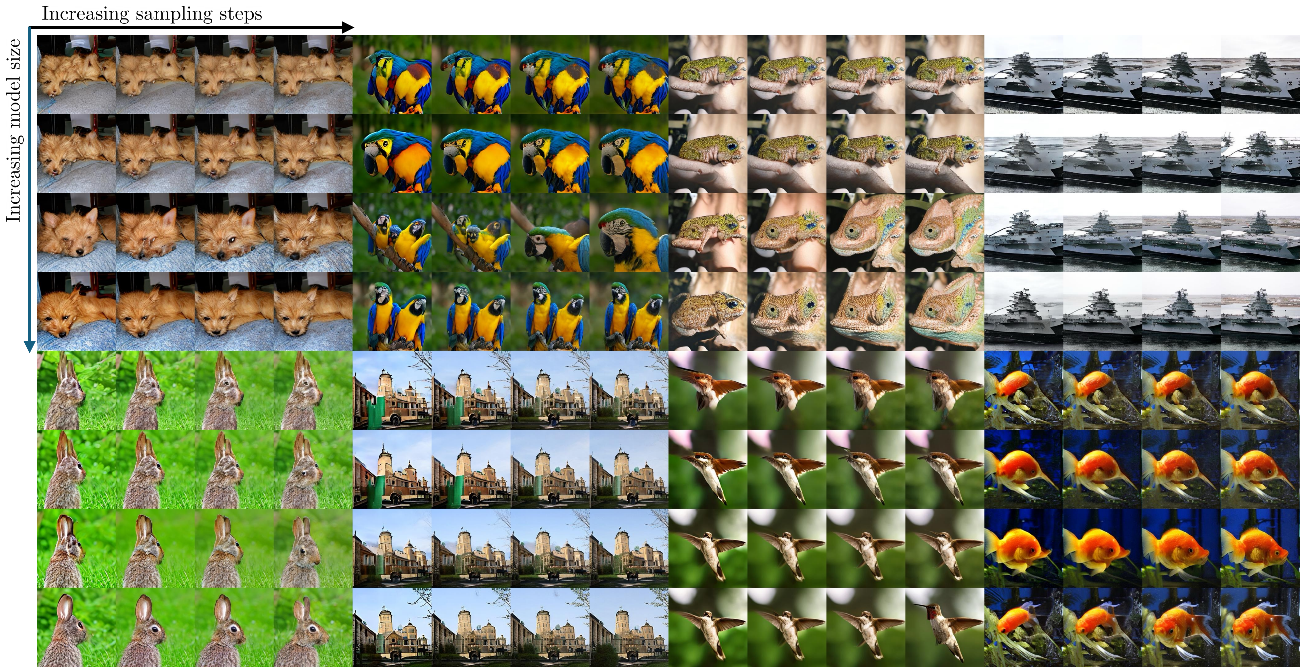

Figure 8. Sample visual quality increases with increase in both model size and sampling compute.

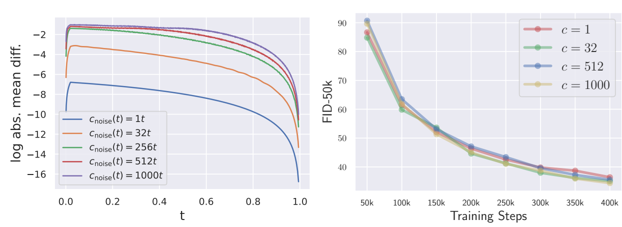

Figure 9. Log distance in embedding space for $c_{\text{noise}}(t) = ct$ (left). Similar ImageNet-256×256 convergence across different c (right).

of M where each group shares the same t. The larger M means that fewer t's are sampled. On the other hand, using extremely small numbers of particles, *e.g.* $M = 2$, leads to training instability and performance degradation, especially on a large scale with DiT architectures. We find that there exists a sweet spot where a few particles effectively help with training stability while further increasing M slows down convergence (see Figure 5). We see that in ImageNet-256×256, training collapses when $M = 1$ (which is CM) and $M = 2$, and achieves lowest FID under the same computation budget with $M = 4$. We hypothesize $M < 4$ does not allow sufficient mixing between particles and larger M means fewer t's are sampled for each step, thus slowing convergence. A general rule of thumb is to use a *large enough* M for stability, but not too large for slowed convergence.

Noise embedding $c_{\text{noise}}(\cdot)$. We plot in Figure 9 the log absolute mean difference of t and $r(s,t)$ in the positional embedding space. Increasing c increases distinguishability of nearby distributions. We also observe similar convergence on ImageNet-256×256 across different c, demonstrating the insensitivity of our framework w.r.t. noise function.

7.3. Sampling

We investigate different sampling settings for best performance. One-step sampling is performed by simple pushforward from T to ϵ (concrete values in Appendix I.2). On CIFAR-10 we use 2 steps and set intermediate time t_1 such that $\eta_{t_1} = 1.4$, a choice we find to work well empirically. On ImageNet-256×256 we go beyond 2 steps and, for simplicity, investigate (1) uniform decrement in t and (2) EDM (Karras et al., 2024) schedule (detailed in Appendix I.2). We plot FID of all sampler settings in Figure 6 with guidance weight $w = 1.5$. We find pushforward sam-

	id/cos	id/FM	sEDM/cos	sEDM/FM	eFM
CIFAR-10	3.77	3.45	2.39	**2.10**	2.53
ImageNet-256×256	46.44	47.32	27.33	28.67	**27.01**

Table 3. FID results with different flow schedules and network parameterization.

plers with uniform schedule to work the best on ImageNet-256×256 and use this as our default setting for multi-step generation. Additionally, we concede that pushforward combined with restart samplers can achieve superior results. We leave such experiments to future works.

7.4. Scaling Behavior

Similar to diffusion models, IMM scale with training and inference compute as well as model size on ImageNet-256×256. We plot in Figure 7 FID vs. training and inference compute in GFLOPs and we find strong correlation between compute used and performance. We further visualize samples in Figure 8 with increasing model size, *i.e.* DiT-S, DiT-B, DiT-L, DiT-XL, and increasing inference steps, *i.e.* 1, 2, 4, 8 steps. The sample quality increases along both axes as larger transformers with more inference steps capture more complex distributions. This also explains that more compute can sometimes yield different visual content from the same initial noise as shown in the visual results.

7.5. Ablation Studies

All ablation studies are done with DDPM++ architecture for CIFAR-10 and DiT-B for ImageNet-256×256. FID comparisons use 2-step samplers by default.

Flow schedules and parameterization. We investigate all combinations of network parameterization and flow schedules: Simple-EDM + cosine (`sEDM/cos`), Simple-EDM + OT-FM (`sEDM/FM`), Euler-FM + OT-FM (`eFM`), Identity + cosine (`id/cos`), Identity + OT-FM (`id/FM`). Identity parameterization consistently fall behind other types of parameterization, which all show similar performance across datasets (see Table 3). We see that on smaller scale (CIFAR-10), `sEDM/FM` works the best but on larger scale (ImageNet-256×256), `eFM` works the best, indicating that OT-FM schedule and Euler paramaterization may be more scalable than other choices.

Mapping function $r(s,t)$**.** Our choices for ablation are (1) constant decrement in η_t, (2) constant decrement in t, (3) constant decrement in $\lambda_t = \log\left(\alpha_t^2/\sigma_t^2\right)$, (4) constant increment in $1/\eta_t$ (see Appendix C.6). For fair comparison, we choose the decrement gap so that the minimum $t - r(s,t)$ is $\approx 10^{-3}$ and use the same network parameterization. FID progression in Figure 11 show that (1) consistently outperforms other choices. We additionally ablate the mapping gap using $M = 4$ in (1). The constant decrement is in

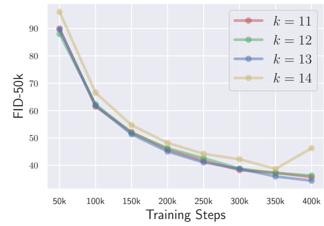

Figure 10. ImageNet-256×256 FID progression with different t, r gap with $M = 4$.

	FID-50k
$w(s,t) = 1$	40.19
+ ELBO weight	96.43
+ α_t	33.44
+ $1/(\alpha_t^2 + \sigma_t^2)$	**27.43**

Table 4. Ablation of weighting function $w(s,t)$ on ImageNet-256×256.

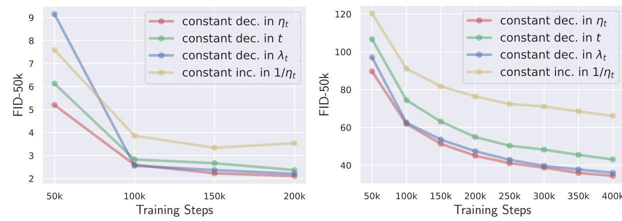

Figure 11. FID progression on different types of mapping function $r(s,t)$ for CIFAR-10 (left) and ImageNet-256×256 (right).

the form of $(\eta_{\max} - \eta_{\min})/2^k$ for an appropriately chosen k. We show in Figure 10 that the performance is relatively stable across $k \in \{11, 12, 13\}$ but experiences instability for $k = 14$. This suggests that, for a given particle number, there exists a largest k for stable optimization.

Weighting function. In Table 4 we first ablate the weighting factors in three groups: (1) the VDM ELBO factors $\frac{1}{2}\sigma(b-\lambda_t)(-\frac{\mathrm{d}}{\mathrm{d}t}\lambda_t)$, (2) weighting α_t (*i.e.* when $a = 1$), and (3) weighting $1/(\alpha_t^2 + \sigma_t^2)$. We find it necessary to use α_t jointly with ELBO weighting because it converts v-pred network to a ϵ-pred parameterization (see Appendix C.9), consistent with diffusion ELBO-objective. Factor $1/(\alpha_t^2 + \sigma_t^2)$ upweighting middle time-steps further boosts performance, a helpful practice also known for FM training (Esser et al., 2024). We leave additional study of the exponent a to Appendix I.5 and find that $a = 2$ emphasizes optimizing the loss when t is small while $a = 1$ more equally distributes weights to larger t. As a result, $a = 2$ achieves higher quality multi-step generation than $a = 1$.

8. Conclusion

We present Inductive Moment Matching, a framework that learns a few-step generative model from scratch. It trains by first leveraging self-consistent interpolants to interpolate between data and prior and then matching all moments of its own distribution interpolated to be closer to that of data. Our method guarantees convergence in distribution and generalizes many prior works. Our method also achieves state-of-the-art performance across benchmarks while achieving orders of magnitude faster inference. We hope it provides a new perspective on training few-step models from scratch and inspire a new generation of generative models.